Практическое занятие №3. Средство мониторинга Nagios

Цель работы

Изучить архитектуру и возможности системы мониторинга Nagios Core. Освоить развёртывание Nagios и агента NRPE с помощью Docker Compose на Debian 12. Получить практические навыки работы с веб-интерфейсом Nagios, создания шаблона хоста, добавления хоста и привязки шаблона, а также настройки проверок сервисов.

Оборудование и программное обеспечение

- Виртуальная машина или физический компьютер с **Debian 12**
- Установленные **Docker Engine** и **Docker Compose** (плагин `docker compose`)
- Доступ к интернету для загрузки Docker-образов
- Браузер для доступа к веб-интерфейсу Nagios
- Минимальные ресурсы: 2 CPU, 2 ГБ ОЗУ, 5 ГБ свободного места на диске

Краткие теоретические сведения

Nagios Core — это свободная система мониторинга IT-инфраструктуры, отслеживающая доступность хостов и сервисов, а также оповещающая администратора при отклонениях. В отличие от Zabbix, Nagios не хранит историю метрик «из коробки» (для этого требуются дополнительные модули, например NagiosGraph), а ориентирован на проверку состояния: «работает / не работает» и пороговые значения.

Ключевые понятия:

- **Хост (Host)** — устройство, которое отслеживается. Определяется директивой `define host`.
- **Сервис (Service)** — проверяемый параметр на хосте (например, PING, HTTP, SSH). Определяется директивой `define service`.
- **Команда (Command)** — вызов плагина Nagios, который выполняет проверку. Определяется директивой `define command`.
- **Шаблон (Template)** — набор общих параметров, который можно наследовать другим хостам или сервисам. В Nagios шаблон — это объект с директивой `register 0`, который не регистрируется как реальный хост, а служит только для наследования.

- **NRPE (Nagios Remote Plugin Executor)** — агент, позволяющий Nagios-серверу запускать плагины на удалённом Linux-хосте. Аналог Zabbix Agent.

Архитектура развёртывания в Docker:

Компонент	Docker-образ	Назначение
Nagios Core	`jasonrivers/nagios:latest`	Сервер мониторинга и веб-интерфейс
NRPE	`mikenowak/nrpe`	Агент для сбора метрик с хоста

Порядок выполнения работы

1. Подготовка окружения

1. Убедитесь, что Docker и Docker Compose установлены:

```
docker --version
docker compose version
```

2. Создайте рабочую директорию и подкаталог для конфигурационных файлов Nagios:

```
mkdir -p ~/nagios-lab/nagios/objects
cd ~/nagios-lab
```

2. Составление файла `compose.yaml`

Создайте файл `compose.yaml` в директории `~/nagios-lab`:

```
services:
  nagios:
    image: jasonrivers/nagios:latest
    container_name: nagios
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      NAGIOSADMIN_USER: nagiosadmin
      NAGIOSADMIN_PASS: nagiosadmin
```

```
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

Пояснения к ключевым параметрам:

- `jasonrivers/nagios` — популярный образ Nagios Core на базе Ubuntu, включающий Nagios, плагины, NRPE и веб-интерфейс.
- `./nagios/objects:/opt/nagios/etc/objects` — монтирование директории с конфигурационными файлами объектов (хосты, сервисы, команды) на хост-машину. Это позволит редактировать конфигурацию снаружи контейнера.
- `NAGIOSADMIN_USER` / `NAGIOSADMIN_PASS` — учётные данные для входа в веб-интерфейс.
- `extra_hosts: host.docker.internal:host-gateway` — позволяет контейнеру Nagios обращаться к хост-машине по имени `host.docker.internal`. Это потребуется, если NRPE работает в режиме `network_mode: host` и слушает на порту 5666 хост-машины.
- `mikenowak/nrpe` — лёгкий NRPE-контейнер на Alpine Linux с набором стандартных плагинов. Работает в режиме `network_mode: host`, чтобы иметь доступ к файловой системе хоста и слушать порт 5666 на хост-машине.
- `NAGIOS_SERVER` — IP-адрес Nagios-сервера, которому разрешено обращаться к NRPE. В данном случае `172.17.0.1` — это адрес docker-шлюза (хост-машины со стороны bridge-сети). При необходимости уточните его командой `ip addr show docker0`.

3. Запуск стека

```
docker compose up -d
```

Проверьте статус:

```
docker compose ps
```

Оба контейнера должны быть в статусе `running`.

4. Вход в веб-интерфейс

1. Откройте браузер и перейдите по адресу `http://<IP-адрес_ВМ>:8080/`.
2. Введите учётные данные:
   - **Username:** `nagiosadmin`
   - **Password:** `nagiosadmin`
3. После входа вы увидите панель **Current Network Status** со списком хостов.

5. Изучение интерфейса Nagios

Ознакомьтесь с основными разделами меню:

Раздел	Назначение
Home	Сводная панель состояния сети
Hosts	Список хостов, их статус и доступность
Services	Список сервисов и их состояние
Problems	Текущие проблемы (сработавшие проверки)
Reports	Отчёты о доступности, уведомлениях, истории
Configuration	Просмотр конфигурации (только чтение в Core)

Задание для самостоятельного изучения: изучите каждый пункт меню, опишите в отчёте, какие данные отображаются в разделах **Hosts** и **Services** для хоста `localhost`, который добавлен по умолчанию.

---

6. Создание простого шаблона мониторинга

Задание 1. Создание шаблона хоста

1. Создайте и откройте директорию `~/nagios-lab/nagios/objects` на хост-машине.
2. Создайте файл `lab-template.cfg` со следующим содержимым:

```
define host {
    name                    lab-linux-template
    use                     generic-host
    notifications_enabled   1
    event_handler_enabled   1
    flap_detection_enabled  1
    process_perf_data       1
    retain_status_information 1
    retain_nonstatus_information 1
    check_command           check-host-alive
    max_check_attempts      5
    notification_interval   30
    notification_period     24x7
    notification_options    d,u,r
```

```
    contact_groups          admins
    register                0
}
```

Пояснение: директива `register 0` делает этот объект шаблоном, а не реальным хостом. Параметр `use generic-host` наследует базовые настройки из встроенного шаблона `generic-host`. Все остальные директивы будут унаследованы хостами, которые используют этот шаблон.

Задание 2. Добавление хоста и привязка шаблона

В том же файле `lab-template.cfg` добавьте определение хоста:

```
define host {
    use                     lab-linux-template
    host_name               my-debian-host
    alias                   My Debian 12 Server
    address                 127.0.0.1
}
```

Пояснение: хост `my-debian-host` использует шаблон `lab-linux-template` через директиву `use`. Адрес `127.0.0.1` указывает на саму хост-машину (в данном случае — на виртуальную машину Debian 12). Если вы хотите мониторить удалённый хост, укажите его IP-адрес.

Задание 3. Добавление сервиса для проверки PING

Добавьте в файл `lab-template.cfg` определение сервиса:

```
define service {
    use                     generic-service
    host_name               my-debian-host
    service_description     PING
    check_command           check_ping!100.0,20%!500.0,60%
}
```

Пояснение: сервис `PING` проверяет доступность хоста. Параметры `100.0,20%!500.0,60%` означают: предупреждение (WARNING), если потери пакетов ≥ 20% или время отклика ≥ 100 мс; критично (CRITICAL), если потери ≥ 60% или время отклика ≥ 500 мс.

Задание 4. Проверка конфигурации

1. Необходимо подмонтировать созданный файл внутрь контейнера nagios. Для этого добавьте следующие строки в файл `compose.yaml`:

```
services:
  nagios:
    image: jasonrivers/nagios:latest
    container_name: nagios
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      NAGIOSADMIN_USER: nagiosadmin
      NAGIOSADMIN_PASS: nagiosadmin
    networks:
      - nagios-net
    extra_hosts:
      - "host.docker.internal:host-gateway"
     # ниже представленны строки, которые надо добавить
     volumes:
      - ./nagios/objects/lab-template.cfg:/opt/nagios/etc/conf.d/lab-
template.cfg
```

2. Пересоздайте контейнеры:

```
docker compose down
docker compose up -d
```

3. Проверьте корректность конфигурации, выполнив внутри контейнера Nagios:

```
docker exec -it nagios /opt/nagios/bin/nagios -v
/opt/nagios/etc/nagios.cfg
```

Убедитесь, что в выводе нет ошибок (строки `Error:` или `Warning:`). Если ошибок нет, перезапустите Nagios:

```
docker restart nagios
```

Задание 5. Проверка мониторинга

1. Обновите веб-интерфейс Nagios. В разделе **Hosts** должен появиться новый хост `my-debian-host`. Дождитесь статуса **OK**.

2. Перейдите в **Services** — должен отображаться сервис `PING` со статусом **OK**.
3. Нажмите на имя сервиса `PING`, чтобы увидеть детали проверки: время последней проверки, статус, вывод плагина.

Задания для самостоятельного выполнения

Вариант 1

1. Добавьте в шаблон `lab-linux-template` дополнительный параметр `notes` с текстом «Учебный шаблон для Debian 12».
2. Создайте нового хоста `my-second-host` с адресом `127.0.0.1`, использующего этот же шаблон.
3. Добавьте сервис `HTTP` с командой `check_http` для проверки веб-сервера на порту 80.
4. Проверьте, что оба хоста отображаются в Nagios и имеют статус **UP**.

Вариант 2

1. Создайте отдельный шаблон `lab-windows-template` с параметрами `check_command check-host-alive` и `max_check_attempts 3`.
2. Добавьте хост `my-windows-host` с адресом `127.0.0.1` (имитация Windows-хоста), использующий этот шаблон.
3. Добавьте сервис `PING` для нового хоста.
4. Сравните параметры наследования: какие значения унаследованы от `generic-host`, а какие переопределены в `lab-windows-template`.

Контрольные вопросы

1. Какие компоненты входят в состав Nagios Core? Опишите их назначение.
2. Чем отличается хост от сервиса в Nagios? Приведите примеры.
3. Что такое шаблон в Nagios? Как создать шаблон и привязать его к хосту?
4. Для чего используется директива `register 0`? Что произойдёт, если её не указать?
5. Как проверить конфигурацию Nagios перед перезапуском? Какая команда для этого используется?

Требования к отчёту

1. **Тема и цель работы.**
2. **Содержимое файла** `compose.yaml` (полный текст).
3. **Скриншоты:**
   - список запущенных контейнеров (`docker compose ps`);
   - главная страница веб-интерфейса Nagios;
   - раздел **Hosts** со всеми хостами (включая созданный `my-debian-host`);
   - раздел **Services** с сервисом `PING` и его деталями;
   - результат проверки конфигурации (`nagios -v`) без ошибок;
4. **Содержимое файла** `lab-template.cfg` (итоговый вариант).
5. **Результаты выполнения индивидуального варианта.**
6. **Выводы:** что освоено, какие трудности возникли, какие навыки приобретены.

Приложение. Полезные команды

```
# Запуск стека
docker compose up -d

# Остановка стека
docker compose stop

# Остановка и удаление контейнеров (данные в volume сохраняются)
docker compose down

# Просмотр логов Nagios
docker compose logs -f nagios

# Проверка конфигурации Nagios
docker exec -it nagios /opt/nagios/bin/nagios -v
/opt/nagios/etc/nagios.cfg

# Перезапуск Nagios после изменения конфигурации
docker restart nagios
```

Важно: при изменении конфигурационных файлов в директории `./nagios/objects` необходимо проверить конфигурацию командой `nagios -v` и перезапустить контейнер `nagios`, чтобы изменения вступили в силу.